

tf.bitwise.bitwise_and Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/bitwise/bitwise_and

Elementwise computes the bitwise AND of x and y.

Function Signatures

```
tf.bitwise.bitwise_and( x: Annotated[Any, TV_BitwiseAnd_T],  
y: Annotated[Any, TV_BitwiseAnd_T], name=None ) ->  
Annotated[Any, TV_BitwiseAnd_T]
```

Code Examples

```
tf.bitwise.bitwise_and(  
x: Annotated[Any, TV_BitwiseAnd_T],  
y: Annotated[Any, TV_BitwiseAnd_T],  
name=None  
) -> Annotated[Any, TV_BitwiseAnd_T]
```

```
tf.bitwise.bitwise_and(  
x: Annotated[Any, TV_BitwiseAnd_T],  
y: Annotated[Any, TV_BitwiseAnd_T],  
name=None  
) -> Annotated[Any, TV_BitwiseAnd_T]
```

```
import tensorflow as tf  
from tensorflow.python.ops import bitwise_ops  
dtype_list = [tf.int8, tf.int16, tf.int32, tf.int64,  
tf.uint8, tf.uint16, tf.uint32, tf.uint64]  
for dtype in dtype_list:  
    lhs = tf.constant([0, 5, 3, 14], dtype=dtype)  
    rhs = tf.constant([5, 0, 7, 11], dtype=dtype)  
    exp = tf.constant([0, 0, 3, 10], dtype=tf.float32)  
    res = bitwise_ops.bitwise_and(lhs, rhs)  
    tf.assert_equal(tf.cast(res, tf.float32), exp) # TRUE
```

```
import tensorflow as tf  
from tensorflow.python.ops import bitwise_ops  
dtype_list = [tf.int8, tf.int16, tf.int32, tf.int64,  
tf.uint8, tf.uint16, tf.uint32, tf.uint64]  
for dtype in dtype_list:  
    lhs = tf.constant([0, 5, 3, 14], dtype=dtype)  
    rhs = tf.constant([5, 0, 7, 11], dtype=dtype)  
    exp = tf.constant([0, 0, 3, 10], dtype=tf.float32)  
    res = bitwise_ops.bitwise_and(lhs, rhs)  
    tf.assert_equal(tf.cast(res, tf.float32), exp) # TRUE
```

Documentation

Elementwise computes the bitwise AND of x and y.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.bitwise.bitwise_and`

The result will have those bits set, that are set in both x and y. The computation is performed on the underlying representations of x and y.

For example:

`## Returns`